Online Learning

Advanced Machine Learning - Master MLDM

Amaury Habrard

amaury.habrard@univ-st-etienne.fr

Laboratoire Hubert Curien, UMR CNRS 5516
Université Jean Monnet Saint-Étienne
amaury.habrard@univ-st-etienne.fr

Semester 1

Outline

Notes

- Slides available on moodle
- References
  - Many sources available on the Web.
  - 2 references used for this course
    - Foundations of Machine Learning. M. Mohri, A. Rostamizadeh, A. Talwalkar. The MIT Press. 2012.
    - Online Learning and Online Convex Optimization. S. Shalev-Shwartz. Foundations and Trends in Machine Learning, vol4, n2, 2011.

Online Learning - Introduction

Online Learning in a few words

Previous PAC Learning models

- Previous models seen so far stand in a batch setting: you learn from a training set and you apply the model on a test set.
- This covers nice algorithms and frameworks
- Test and training data follow the same distribution
- Not adapted to large scale problems.

Online Learning

- Considers one example at a time
- Attractive solution for large-scale problems, efficient both in time and space, easy to implement
- No distributional assumption is made
- Performance of online algorithms is measured using a mistake model and the notion of regret.
- Theoretical guarantees are based on worst-case settings or adversarial assumptions.

Online Learning Setting

- Domain set (feature space), $\mathcal{X}$ (often $\subseteq \mathbb{R}^d$)
- Label set $\mathcal{Y}$: set of possible labels, here mainly $\{-1, 1\}$.
- A prediction rule, $h : \mathcal{X} \rightarrow \mathcal{Y}$ used to label future examples. This function is called a predictor, a hypothesis or a classifier.
- A loss function $l : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$.
- Labels are assumed to be assigned with respect to f that comes from some hypothesis class $\mathcal{H} \subset \mathcal{Y}^{\mathcal{X}}$.
  $\mathcal{H}$ is the hypothesis class considered.

A Generic Algorithm

for $t \leftarrow 1$ *to* T **do**

Environment presents an instance $x_t \in \mathcal{X}$;

Learner predicts label $\hat{y}_t \in \mathcal{Y}$ with the current model;

Learner receives the true label $y_t \in \mathcal{Y}$;

Learner updates the model if $y_t \neq \hat{y}_t$ (or/and pay the cost defined by the associated loss);

Learning is difficult if no more assumption are made:

- Imagine an adversarial environment where the true label is always different from the prediction made: no learning is possible

Type of bounds

Mistake bounds

Corresponds to the maximum number of mistakes a learning algorithm L can make over a sequence of T examples to learn a concept c.

$$\text{mistakes}(L, c) = \max_{\mathbf{x}_1,\ldots,\mathbf{x}_T} \sum_{t=1}^{T} [L(x_t) \neq c(x_t)] = \max_{\mathbf{x}_1,\ldots,\mathbf{x}_T} \sum_{t=1}^{T} l(L(x_t), c(x_t)).$$

(Can be extended to maximum over the class of concepts)

Regret

Difference of total loss and the one of the best expert

$$R_T = \sum_{t=1}^{T} l(\hat{y}_t, y_t) - \min_{1 \leq i \leq N} \sum_{t=1}^{T} l(\hat{y}_{t,i}, y_t).$$

Online Learning with Finite Classes

Learning with Finite classes

$\mathcal{H}$ is of finite class - Examples

- All the functions in $\mathcal{Y}^{\mathcal{X}}$ that can be implemented by a Python program of length at most b.
- finite set of threshold in $[0,1]$ from a finite set $\{0, 1/n, 2/n, \ldots, 1\}$, ...
- Realizable case: there exists an hypothesis consistent with the data

The consistent learner

Initialize $V_1 \leftarrow \mathcal{H}$;
for $t \leftarrow 1$ *to* T **do**
  Get $x_t \in \mathcal{X}$;
  Pick some $h \in V_t$ at random;
  Predict label $\hat{y}_t = h(x_t)$;
  Get $y_t \in \mathcal{Y}$;
  update $V_{t+1} = \{h \in V_t : h(x_t) = y_t\}$;

Analysis

Theorem

The consistent learner will make at most $|\mathcal{H}| - 1$ mistakes.

Proof

If we have an error at step t, then the h_t used for prediction will not be in V_{t+1}, therefore $|V_{t+1}| \leq |V_t| - 1$

Can we do better?

Analysis - Mistake bound

Theorem

Let $\mathcal{H}$ a finite hypothesis class, $f \in \mathcal{H}$ and let $\{(\boldsymbol{x}_1, f(\boldsymbol{x}_1)), \ldots, (\boldsymbol{x}_T, f(\boldsymbol{x}_T))\}$ an arbitrary sequence of examples. Then, the expected number of mistakes made by the algorithm is at most $\log(|\mathcal{H}|)$.

Proof

For each round t, let $\alpha_t = |V_{t+1}|/|V_t|$. Therefore, after T rounds we have

$$1 \leq |V_{T+1}| = |\mathcal{H}| \prod_{t=1}^{T} \alpha_t.$$

Using the inequality $b \leq e^{-(1-b)}$, which holds for all b, we get that

$$1 \leq |\mathcal{H}| \prod_{t=1}^{T} e^{-(1-\alpha_t)} = |\mathcal{H}| e^{-\sum_{t=1}^{T}(1-\alpha_t)}$$

which implies $\sum_{t=1}^{T}(1-\alpha_t) \leq \ln(|\mathcal{H}|)$. (1)

Proof continued

Finally, since we predict $\hat{y}_t$ by choosing $h \in V_t$ uniformly at random, we have that the probability to make a mistake on round t is:

$$P[\hat{y}_t \neq y_t] = \frac{|\{h \in V_t : h(\boldsymbol{x}_t) \neq y_t\}|}{|V_t|} = \frac{|V_t| - |V_{t+1}|}{|V_t|} = (1 - \alpha_t).$$

Therefore the expected number of mistakes is :

$$\sum_{t=1}^{T} \mathbb{E}[\mathbf{1}_{\hat{y}_t \neq y_t}] = \sum_{t=1}^{T} P[\hat{y}_t \neq y_t] = \sum_{t=1}^{T} (1 - \alpha_t) \leq \ln(|\mathcal{H}|)$$

Last inequality is obtained by the Inequality (1) of previous slide.

The Halving learner

The Halving learner

Initialize $V_1 \leftarrow \mathcal{H}$;
for $t \leftarrow 1$ *to* T **do**
 Get $x_t \in \mathcal{X}$;
 Predict $\hat{y}_t = \text{Majority}\{h(x_t) | h \in V_t\}$;
 Get $y_t \in \mathcal{Y}$;
 if $\hat{y}_t \neq y_t$ **then**
  update $V_{t+1} = \{h \in V_t : h(x_t) = y_t\}$;

Analysis - Mistake bound

Theorem

The Halving learner will make at most $\log_2(|\mathcal{H}|)$ mistakes.

Proof

If we have an error at step t, then at least half of the functions in V_t will make an error and will not be in V_{t+1}. Therefore $|V_{t+1}| \leq |V_t|/2$ (hence the name Halving) and thus if M is the number of mistakes:
$1 \leq |V_{T+1}| \leq |\mathcal{H}|2^{-M}$. Rearranging the last inequality concludes the proof.

Corollary

The Halving learner can learn the class $\mathcal{H}$ of all pythons programs of length $< b$ bits while making at most b mistakes.

We do not take into account the running time, we assume $f \in \mathcal{H}$, ...

Optimal mistake bound

Let $Opt(\mathcal{H})$ the optimal mistake bound for $\mathcal{H}$.

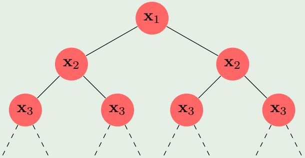
theorem

$VCdim(\mathcal{H}) \leq Opt(\mathcal{H}) \leq \text{Mistake}_{\text{Halving}}(\mathcal{H}) \leq \log_2(\mathcal{H})$

Recaps

VC dimension of a class $\mathcal{H}$, denoted $VCdim(\mathcal{H})$, is the maximal number d such that there are instances $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_d$ that are shattered by $\mathcal{H}$. That is, for any sequence of labels $(y_1, \ldots, y_d) \in \mathcal{Y}^d$ there exists a hypothesis $h \in \mathcal{H}$ that gives exactly this sequence of labels.

Below a complete binary tree (left: positive, right: negative):



Optimal mistake bound

Let $Opt(\mathcal{H})$ the optimal mistake bound for $\mathcal{H}$.

theorem

$VCdim(\mathcal{H}) \leq Opt(\mathcal{H}) \leq \text{Mistake}_{\text{Halving}}(\mathcal{H}) \leq \log_2(\mathcal{H})$

Proof

Second inequality is true by definition and third corresponds to the theorem shown previously.
For the 1st inequality, let $d = VCdim(\mathcal{H})$. Then there exists a shattered set of d points for which we can form a complete binary tree of the mistakes with height d, and we can choose labels at each round of learning to ensure that d mistakes are made. By definition of VC dimension, this situation exists. This adversarial argument is valid since the online setting makes no statistical assumptions about the data.

So far we were in the realizable setting, we now check the unrealizable one.

Weighted Majority Algorithm (WM)

$\{h_1, \ldots, h_N\}$ finite set of experts/classifiers, T:nb of rounds, $\beta \in [0, 1]$

```
for i ← 1 to N do w_{1,i} ← 1;
for t ← 1 to T do
    Receive x_t;
    W_t ← Σ_{i=1}^N w_{t,i};
    if Σ_{i=1}^N (w_{t,i}/W_t) h_i(x_t) ≥ 0 then ŷ_t ← 1;
    else ŷ_t ← −1;
    Receive y_t;
    if y_t ≠ ŷ_t then
        for i ← 1 to N do
            if h_i(x_t) ≠ y_t then w_{t+1,i} ← β w_{t,i};
            else w_{t+1,i} ← w_{t,i};
return the weighted majority given by the weights at T + 1;
```

Analysis Mistake bound

Theorem

Fix $\beta \in (0,1)$. Let m_T be the number of mistakes made by algorithm after $T \geq 1$ and let m_T^* be the number of mistakes by the best of the N experts. Then the following inequality holds:

$$m_T \leq \frac{\log(N) + m_T^* \log(1/\beta)}{\log(2/(1+\beta))}.$$

This bound offers the following type of guarantees:

$$m_T \leq O(\log(N)) + \text{constant} \times |\text{ mistakes of the best expert }|.$$

- Since the first term varies logarithmically as a function of N, the theorem guarantees that the number of mistakes is roughly a constant times the best expert in hindsight.
- Strong result because we have no assumption about the sequence of points nor the labels, in particular the sequence could be chosen adversarially.
- In the realizable case, $m_T^* = 0$, the bound reduces to $m_T \leq O(\log N)$ as for the Halving algo.

Proof

For any $t \geq 1$, we define our potential function as $W_t = \sum_{i=1}^{N} w_{t,i}$. Since predictions are generated by a weighted majority vote, if the algorithm makes an error at round t, this implies that:

$$W_{t+1} \leq (1/2 + (\beta/2))W_t = \left(\frac{1+\beta}{2}\right) W_t$$

Since $W_1 = N$ and m_T mistakes are made after T rounds, we thus have the following upper bound

$$W_T \leq \left(\frac{1+\beta}{2}\right)^{m_T} N.$$

Proof continued

Now, the weights are non negative and it is clear that for any expert i, $W_T \geq w_{t,i} = \beta^{m_{T,i}}$ where $m_{T,i}$ is the number of mistakes made by the ith expert after T rounds. Applying this lower bound to the best expert and combining it with the previous upper bound, we have

$$\beta^{m_T^*} \leq W_T \leq \left(\frac{1+\beta}{2}\right)^{m_T} N$$
$$\Rightarrow m_T^* \log(\beta) \leq \log(N) + m_T \log\left(\frac{1+\beta}{2}\right)$$
$$\Rightarrow m_T \log\left(\frac{2}{1+\beta}\right) \leq \log(N) + m_T^* \log(\frac{1}{\beta})$$

Pb with deterministic algorithms

- No deterministic algo can achieve a regret $o(T)^1$ over all sequences
  - Given a deterministic algo A, at any t we can adversarially select y_t to be 1 if A predicts -1 and choose -1 otherwise. Thus A errs at every point of the sequence and thus $m_T = T$.
  - Assume $N = 2$ and that one expert predicts always -1 and the other always 1. The error of the best expert is at most $m_T^* \leq T/2$, thus for that sequence we have
  $$R_T = m_T - m_T^* \geq T/2$$
- This negative result does not hold for convex losses in at least of of its argument
- But for the 0-1 loss, we must consider a randomized version of the algorithm.

[1] $f(n) \in o(g(n))$: $\forall \epsilon > 0, \exists n_0 \ s.t. \ \forall n > n_0, |f(n)| \leq |g(n)| \cdot \epsilon$

Randomized weighted majority vote (RWM)

$\mathcal{H} = \{h_1, \ldots, h_N\}$ finite set of experts (classifiers), T number of rounds.

for $i \leftarrow 1$ *to* N **do** $w_{1,i} \leftarrow 1$;
for $t \leftarrow 1$ *to* T **do**
  Let $Z_t = \sum_{i=1}^{N} w_{t;i}$;
  sample an a $h \in \mathcal{H}$ at random according to $(\frac{w_{t,1}}{Z_t}, \ldots, \frac{w_{t,d}}{Z_t})$;
  Receive $\boldsymbol{x}_t$;
  Predict $\hat{y}_t = h(\boldsymbol{x}_t)$;
  Receive y_t;
  for $i \leftarrow 1$ *to* N **do**
    if $h_i(\boldsymbol{x}_t) \neq y_t$ **then** $w_{t+1,i} \leftarrow \beta w_{t,i}$;
    else $w_{t+1,i} \leftarrow w_{t,i}$;
return the weighted majority given by the weights at $T+1$;

Analysis - Regret bound

Theorem

Fix $\beta \in (1/2, 1)$. Then, for any $T \geq 1$ the loss of algorithm RWM on any sequence can be bounded as follows:

$$L_T \leq \frac{\log(N)}{1-\beta} + (2-\beta)L_T^{min}.$$

In particular, for $\beta = \max(1/2, 1 - \sqrt{\log(N)/T})$, we have

$$L_T \leq L_T^{min} + 2\sqrt{T\log(N)}.$$

This algorithm uses the 0-1 loss, we denote by $l_{t,i}$ the value of the loss for the classifier i at step t. Let $p_{t,i} = \frac{w_{t,i}}{Z_t}$.

The expected loss at round t is: $L_t = \sum_{i=1}^{N} p_{t,i} l_{t,i}$. The total loss over T rounds is defined as $\mathcal{L}_T = \sum_{t=1}^{T} L_t$. The loss for a classifier i is $\mathcal{L}_{T,i}$. The minimal loss is the minimal loss over the classifier set: $\mathcal{L}_T^{min} = \min_{1\leq i\leq N}\mathcal{L}_{T,i}$.

The regret after T rounds is defined as: $R_T = \mathcal{L}_T - \mathcal{L}^{min}$.

Proof

We proceed similarly as the previous proof. Let $W_t = \sum_{i=1}^{N} w_{t,i} (= Z_t)$.
By definition of the algorithm

$$
\begin{aligned}
W_{t+1} = \sum_{i, l_{t,i}=0} w_{t,i} + \beta \sum_{i, l_{t,i}=1} w_{t,i} &= W_t + (\beta - 1) \sum_{i, l_{t,i}=1} w_{t,i} \\
&= W_t + (\beta - 1) W_t \sum_{i, l_{t,i}=1} p_{t,i} \\
&= W_t + (\beta - 1) W_t L_t \\
&= W_t (1 - (1 - \beta) L_t).
\end{aligned}
$$

Thus since $W_1 = N$, it follows that $W_{T+1} = N \prod_{t=1}^{T} (1 - (1 - \beta) L_t)$.
The lower bound clearly holds: $W_{T+1} \geq \max_{i \in [1,N]} w_{T+1,i} = \beta^{\mathcal{L}_T^{min}}$.

Proof continued

We need the following inequalities: $\log(1-x) \leq -x$ for all $x < 1$ and $-\log(1-x) \leq x + x^2$ for $x \in [0, 1/2]$:

$$
\begin{aligned}
\beta^{\mathcal{L}_T^{min}} &\leq N \prod_{t=1}^{T} (1 - (1-\beta) L_t) \\
&\Rightarrow \mathcal{L}_T^{min} \log \beta \leq \log(N) + \sum_{t=1}^{T} \log(1 - (1-\beta) L_t) \\
&\Rightarrow \mathcal{L}_T^{min} \log \beta \leq \log(N) - (1-\beta) \sum_{t=1}^{T} L_t \\
&\Rightarrow \mathcal{L}_T^{min} \log \beta \leq \log(N) - (1-\beta) \mathcal{L}_T \\
&\Rightarrow \mathcal{L}_T \leq \frac{\log(N)}{1-\beta} - \frac{\log \beta}{1-\beta} \mathcal{L}_T^{min} \\
&\Rightarrow \mathcal{L}_T \leq \frac{\log(N)}{1-\beta} - \frac{\log(1-(1-\beta))}{1-\beta} \mathcal{L}_T^{min} \\
&\Rightarrow \mathcal{L}_T \leq \frac{\log(N)}{1-\beta} + (2-\beta) \mathcal{L}_T^{min}
\end{aligned}
$$

Proof continued

This shows the first statement. Since $\mathcal{L}_T^{min} \leq T$ this also implies:

$$\mathcal{L}_T \leq \frac{\log(N)}{1-\beta} + (1-\beta)T + \mathcal{L}_T^{min}.$$

Differentiating the upper bound with respect to β and setting it to zero gives $\frac{\log(N)}{(1-\beta)^2} - T = 0$, that is $\beta = \beta_0 = 1 - \sqrt{\log(N)/T}$, since $\beta < 1$.
Thus if $1 - \sqrt{\log(N)/T} \geq 1/2$, β_0 is the minimizing value of β otherwise $1/2$ is the optimal value.
The second statement follows by replacing β with β_0 in the inequality at the top of the page.

Analysis - Discussion

The bound we showed assumes the algorithm receives T as a parameter.

$$L_T \leq L_T^{min} + 2\sqrt{T \log(N)}.$$

There exists a little trick allowing to relax this requirement at the price of a small constant.
The bound above can be written directly in terms of regret of the RWM algorithm:

$$R_T \leq 2\sqrt{T \log(N)}.$$

Thus for N constant, the regret verifies $R_T = O(\sqrt{T})$ and the average regret or regret per round R_T/T decreases as $O(1/\sqrt{T})$
These results are actually optimal as shown in the next theorem.

Optimality theorem

Theorem

Let $N = 2$. There exists a stochastic sequence of losses for which the regret of any online algorithm verifies $\mathbb{E}[R_T] \geq \sqrt{T/8}$.

Proof

For any $t \in [1, T]$, let the vector of losses $\boldsymbol{l}_t = (l_{t,1}, l_{t,2})^T$ take the values $(0,1)^T$ and $(1,0)^T$ with equal probability. Then, the expected loss of any randomized algorithm is

$$\mathbb{E}\mathcal{L}_T = \mathbb{E}[\sum_{t=1}^{T} \boldsymbol{p}_t \boldsymbol{l}_t] = \sum_{t=1}^{T} \boldsymbol{p}_t \mathbb{E}[\boldsymbol{l}_t] = \sum_{t=1}^{T} \frac{1}{2} p_{t,1} + \frac{1}{2}(1 - p_{t,1}) = T/2$$

$\boldsymbol{p}_t$ is the distribution considered by the algo at step t.

Optimality theorem - Proof continued

By definition of $\mathcal{L}_T^{min}$, we have:

$$\begin{aligned}\mathcal{L}_T^{min} =& \min(\mathcal{L}_{T,1}, \mathcal{L}_{T,2})\\ =& \frac{1}{2}(\mathcal{L}_{T,1} + \mathcal{L}_{T,2} - |\mathcal{L}_{T,1} - \mathcal{L}_{T,2}|) = T/2 - |\mathcal{L}_{T,1} - T/2|\end{aligned}$$

using the fact that $|\mathcal{L}_{T,1} + \mathcal{L}_{T,2}| = T$.

Thus the expected regret of the algorithm is:

$$\mathbb{E}[R_T] = \mathbb{E}[\mathcal{L}_T] - \mathbb{E}[\mathcal{L}_T^{min}] = \mathbb{E}|\mathcal{L}_{T,1} - 1/2|$$

Optimality theorem - Proof continued

Let $\sigma_t, t \in [1, T]$ be Rademacher variables taking values in $\{-1, 1\}$ (*i.e.* each variable has a probability of 0.5 to be 1 and 0.5 to be -1), then $\mathcal{L}_{T,1}$ can be rewritten as

$$\mathcal{L}_{T,1} = \sum_{t=1}^{T} \frac{1+\sigma}{2} = T/2 + \frac{1}{2}\sum_{t=1}^{T} \sigma_t.$$

Introducing scalars $x_t = 1/2, t \in [1, T]$, by the Khintchine-Kahane inequality, we have:

$$\mathbb{E}[R_T] = \mathbb{E}\left[|\sum_{t=1}^{T} \sigma_t x_t| \right] \geq \sqrt{\frac{1}{2}\sum_{t=1}^{T} x_t^2} = \sqrt{T/8}.$$

More generally, for $T \geq N$, a lower bound for R_T in $\Omega(\sqrt{T \log(N)})$ can be proven.

Khintchine-Kahane inequality:
$\frac{1}{2}\sum_{i=1}^{m} \|\boldsymbol{x}_i\|^2 \leq \left(\mathbb{E}_{\boldsymbol{\sigma}}\left[\|\sum_{i=1}^{m} \sigma_i \boldsymbol{x}_i\|\right]\right)^2 \leq \sum_{i=1}^{m} \|\boldsymbol{x}_i\|^2$

For other losses

Setting for other losses

- We consider general loss functions in $[0, 1]$ that are convex in their first argument.
- An extension with Exponential Weighted Average proposes a solution in this context
- The algorithm prediction $\hat{y}_t = \frac{\sum_{i=1}^{N} w_{t,i} h_i(\mathbf{x})}{\sum_{i=1}^{N} w_{t,i}}$
- The algorithm updates the weight at the end of each turn:
$$w_{t+1,i} \leftarrow w_{t,i} e^{-\eta l(h_t(\mathbf{x}), y_t)} = e^{-\eta L_{t,i}}$$

Associated results (1/2)

Theorem

Assume the loss function L is convex in its first argument and takes values in $[0,1]$. Then for any $\eta > 0$ and any sequence of labels $y_1, \ldots, y_T \in Y$, the regret of the algo satisfies after T rounds:

$$R_T \leq \frac{\log(N)}{\eta} + \frac{\eta T}{8}$$

In particular for $\eta = \sqrt{8 \log(N)/T}$, the regret becomes:

$$R_T \leq \sqrt{(T/2)\log(N)}$$

Associated results (2/2)

The previous theorem requires to know T to define η, by dividing time into periods of size 2^k we can get rid of the constraint on η:

Theorem

With the same assumptions as the previous theorem but without η we have:

$$R_T \leq \frac{\sqrt{2}}{\sqrt{2}-1}\sqrt{(T/2)\log(N)} + \sqrt{\log(N)/2}.$$

Learning Linear Models

Learning Linear methods

Perceptron algorithm - recap simplified version

Input: Training set T, learning rate $\eta > 0$
begin
  $k \leftarrow 0$, $w_k \leftarrow \vec{\mathbf{0}}$, $R \leftarrow \max_{i=1}^{n} \|\mathbf{x_i}\|_2$
  for $i \leftarrow 1$ **to** T **do**
    if $y_i(\langle \mathbf{w_k}, \mathbf{x_i} \rangle) \leq 0$ **then**
      $\mathbf{w_{k+1}} \leftarrow \mathbf{w_k} + y_i \mathbf{x_i}$ //more generally $\eta y_i \mathbf{x_i}$
      $k \leftarrow k + 1$
  return k, w_k,

We return: $h(\mathbf{x}) = \sum_{i \in I} y_i \langle \mathbf{x_i} \cdot \mathbf{x} \rangle$ where is I is the subset of the T rounds where where an update has been made and $\mathbf{x_i}$ is the associated learning example. Points from I can be considered as support vectors.

Analysis - Mistake bound

Theorem

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T \in \mathbb{R}^d$ be a sequence of T points such that $\|\boldsymbol{x}_t\| \leq R$. Assume there exits $\gamma > 0$ and $\boldsymbol{w} \in \mathbb{R}^d$ such that for all $t \in [1, T]$, $\gamma < \frac{y_t \langle \mathbf{w}, \mathbf{x_t} \rangle}{\|\boldsymbol{w}\|}$. Then, the number of updates made by the perceptron algorithm si bounded by R^2/γ^2.

- Here we assume the data to be separable
- The bound is remarkable since it depends only on the normalized margin γ/R and not on the dimension of the space.
- This bound can be shown to be tight: the number of updates can be equal to R^2/γ^2 in some instances.
- We can also deduce generalization guarantees that will depend on the M, the sample and its size, and the (squared) normalized margin.

Pre-requisite

Cauchy-Schwarz Inequality (CSI)

$$|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\| \cdot \|\mathbf{y}\|$$

Recall the updates

- $\mathbf{w_{k+1}} = \mathbf{w_k} + y_i \mathbf{x_i}$
- Thus $\mathbf{w_{k+1}} - \mathbf{w_k} = y_i \mathbf{x_i}$.
- Algo converges if the data are linearly separable
- Number of iterations depends on the margin

Perceptron - proof of mistake bound

Proof

Let I be the subset of the T rounds where an update is made and let M be the total number of updates ($|I| = M$). Summing up assumption inequalities and using CSI yields:

$$M\gamma \leq \frac{\langle \boldsymbol{w}, \sum_{t\in I} y_t \boldsymbol{x}_t \rangle}{\|\boldsymbol{w}\|} \leq \|\sum_{t\in I} y_t \boldsymbol{x}_t\| = \|\sum_{t\in I} \boldsymbol{w}_{t+1} - \boldsymbol{w}_t\| = \|\boldsymbol{w}_{M+1}\|$$

$$\text{(telescoping, } w_0 = 0\text{)} \qquad = \sqrt{\sum_{t\in I} \|\boldsymbol{w}_{t+1}\|^2 - \|\boldsymbol{w}_t\|^2}$$

$$\text{(updates)} \qquad = \sqrt{\sum_{t\in I} \|\boldsymbol{w}_t + y_t \boldsymbol{x}_t\|^2 - \|\boldsymbol{w}_t\|^2}$$

$$(2y_t \boldsymbol{w}_t \cdot \boldsymbol{x}_t \leq 0) \qquad = \sqrt{\sum_{t\in I} 2y_t \boldsymbol{w}_t \cdot \boldsymbol{x}_t + \|\boldsymbol{x}_t\|^2}$$

$$\leq \sqrt{\sum_{t\in I} \|\boldsymbol{x}_t\|^2} \leq \sqrt{MR^2}.$$

Comparing left- and right-hand sides, we get $\sqrt{M} \leq R/\gamma$ and thus $M \leq R^2/\gamma^2$.

Perceptron

Remarks

- The result is remarkable: it does not depend on the dimension of the space.
- It can be shown to be tight: the bound can be achieved in some particular instances
- The result does not depend on the sequence of learning examples.

Generalization bound

Let $M(S)$ be the number of updates made by the algorithm after training over S of size m and let h_S be the hypothesis returned, r_S is the radius of the smallest ball containing S and ρ_S the highest margin of an hyperplane w.r.t. S, then the expected error w.r.t. 0-1 loss is bounded as follows:

$$\mathbb{E}_{S\sim D^m}[R(h_S)] \leq \mathbb{E}_{S\sim D^{m+1}}\left[\frac{min(M(S), r_S^2/\rho_S^2)}{m+1}\right]$$

Perceptron - proof of generalization bound

Proof

Let S be a linearly separable sample of size $m+1$ drawn i.i.d. according to D and let $\boldsymbol{x}$ a point in S. If $h_{S\backslash\{\boldsymbol{x}\}}$ misclassifies $\boldsymbol{x}$ then $\boldsymbol{x}$ must be a support vector for h_S, otherwise $h_{S\backslash\{\boldsymbol{x}\}} = h_S$, then the leave-one-out error is then at most $M(S)/(m+1)$. The we have for the leave-one-out error:

$$\begin{aligned}
\frac{1}{m+1}\sum_{i=1}^{m+1}\mathbb{E}_{S\sim D^{m+1}}[1_{h_{S\backslash\{\boldsymbol{x}_i\}}(\boldsymbol{x}_i)\neq y_i}] &= \mathbb{E}_{S\sim D^{m+1}}[1_{h_{S\backslash\{\boldsymbol{x}_1\}}(\boldsymbol{x}_1)\neq y_1}] \\
&= \mathbb{E}_{S'\sim D^m, \boldsymbol{x}_1\sim D}[1_{h_{S'}(\boldsymbol{x}_1)\neq y_1}] \\
&= \mathbb{E}_{S'\sim D^m}[\mathbb{E}_{\boldsymbol{x}_1\sim D}[1_{h_{S'}(\boldsymbol{x}_1)\neq y_1}]] \\
&= \mathbb{E}_{S'\sim D^m}[R(h_{S'})]
\end{aligned}$$

$\rightarrow$ linearity of expectation, i.i.d. aspect of expectation does not depend on index i. Combining this result with the previous thm leads to the result.

Stochastic Gradient Descent

Stochastic Gradient Descent - SGD

- When one aims at optimizing a (regularized) empirical loss over a sample of n examples, SGD is a good alternative when n is large.

1- Sample an exemple i at random;
2- Update parameter: $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta_t \nabla \ell((\boldsymbol{x}_i, y_i), \boldsymbol{w}_t)$;
3- return to step 1 until a stopping criterion is met.;

- In expectation, we are moving to the opposite direction of the gradient
- Learning rate must be chosen with care
- Notations $L(\boldsymbol{w}) = \frac{1}{n}\sum_i \ell((\boldsymbol{x}_i, y_i), \boldsymbol{w})$, $\boldsymbol{w}_*$ best model.

Theorem

Suppose $\|\nabla\ell((\boldsymbol{x}, y), \boldsymbol{w})\| \leq B$ (norm of gradient is bounded)

suppose we know a bound on our starting distance: $\|\boldsymbol{w}_0 - \boldsymbol{w}_*\| \leq R$, where w_0 is the starting point.

Set $\eta = \frac{R}{B}\sqrt{\frac{2}{T}}$, then we have:

$$\mathbb{E}[L(\bar{\boldsymbol{w}}_T)] - L(\boldsymbol{w}_*) \leq \frac{RB}{\sqrt{T}} \text{ where } \bar{\boldsymbol{w}}_T = \frac{1}{T}\sum_i \boldsymbol{w}_t$$

Proof

- Suppose $(\boldsymbol{x}_i, y_i)$ is drawn at timestep t. Let us define sampled loss function at time t to be: $\ell_t(\boldsymbol{w}) = \ell((\boldsymbol{x}_i, y_i), \boldsymbol{w})$.
- we have that

$$
\begin{aligned}
\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_*\|^2 &= \|\boldsymbol{w}_t - \eta \nabla \ell_t(\boldsymbol{w}_t) - \boldsymbol{w}_*\|^2 \\
&= \|\boldsymbol{w}_t - \boldsymbol{w}_*\|^2 - 2\eta \nabla \ell_t(\boldsymbol{w}_t) \cdot (\boldsymbol{w}_t - \boldsymbol{w}_*) + \eta^2 \|\nabla \ell_t(\boldsymbol{w}_t)\|^2 \\
&\leq \|\boldsymbol{w}_t - \boldsymbol{w}_*\|^2 - 2\eta \nabla \ell_t(\boldsymbol{w}_t) \cdot (\boldsymbol{w}_t - \boldsymbol{w}_*) + \eta^2 B^2
\end{aligned}
$$

Proof

- Due to random sampling at time t (uncorrelated with the history):

$$\mathbb{E}[\nabla \ell_t(\boldsymbol{w}_t) | \text{ history before } t] = \nabla L(\boldsymbol{w}_t)$$

- By taking expectation with respect to a sample at time t:

$$\mathbb{E}[\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_*\|^2] \leq \mathbb{E}[\|\boldsymbol{w}_t - \boldsymbol{w}_*\|^2 - 2\eta \nabla L(\boldsymbol{w}_t) \cdot (\boldsymbol{w}_t - \boldsymbol{w}_*)] + \eta^2 B^2$$

- We have:

$$\mathbb{E}[\nabla L(\boldsymbol{w}_t) \cdot (\boldsymbol{w}_t - \boldsymbol{w}_*)] \leq \mathbb{E}[\frac{1}{2\eta}\|\boldsymbol{w}_t - \boldsymbol{w}_*\|^2] - \mathbb{E}[\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_*\|^2] + \frac{\eta}{2} B^2$$

Proof

- and then, using the proposed η

$$\mathbb{E}[\frac{1}{T}\sum_{t=1}^{T}\nabla L(\boldsymbol{w}_t)\cdot(\boldsymbol{w}_t-\boldsymbol{w}_*)] = \frac{1}{2\eta}\mathbb{E}[\|\boldsymbol{w}_t-\boldsymbol{w}_*\|^2-\|\boldsymbol{w}_{T+1}-\boldsymbol{w}_*\|^2]+\frac{\eta}{2}B^2$$

$$\leq \frac{\|\boldsymbol{w}_t-\boldsymbol{w}_*\|^2}{2\eta}+\frac{\eta}{2}B^2$$

since $\|\boldsymbol{w}_{T+1}-\boldsymbol{w}_*\|^2 \geq 0$, then with $t=1$,

using update rule and triangle inequality, we get:

$$\leq \frac{(\|\eta\nabla L(\boldsymbol{w}_0)\|^2+\|\boldsymbol{w}_0-\boldsymbol{w}_*\|^2)}{2\eta}+\frac{\eta}{2}B^2$$

$$\leq \frac{RB}{\sqrt{T}} \quad \text{using } \eta=\frac{R}{B}\sqrt{\frac{2}{T}}$$

- Proof is finished by using convexity: $f(x)\geq f(y)+\nabla f(y)\cdot(x-y)$

Generalization bound

- We can apply the same technique with $L(\boldsymbol{w}) = \mathbb{E}[\ell((\boldsymbol{x}, y), \boldsymbol{w})]$ to obtain:
$$\mathbb{E}[L(\bar{\boldsymbol{w}}_T)] - L(\boldsymbol{w}_*) \leq \frac{RB}{\sqrt{T}}$$
- It provides a *generalization bound* in that it bounds the risk of the true loss.

Online to Batch Conversions

On-line to batch conversion (1/2) [Mohri et al., 2012]

There are theorems that give guarantees on algorithm processing learning samples of size T by an online algorithm.

theorem

Let $S = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^T$ be a labeled sample drawn i.i.d. from a distribution D and $L(\cdot,\cdot)$ is a loss function bounded by M and $h_1, \ldots h_T$ be the sequence of models learned by an online algorithm. Then, for any $\delta > 0$, with probability at least $1-\delta$:

$$\frac{1}{T}\sum_{i=1}^{T} R(h_i) \leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(\boldsymbol{x}_i), y_i) + M\sqrt{\frac{2\log(1/\delta)}{T}}$$

where $R(h) = \mathbb{E}_{(\boldsymbol{x},y)\sim D} L(h(\boldsymbol{x}), y)$

On-line to batch conversion (2/2) [Mohri et al., 2012]

Theorem

Under the same assumptions as the previous theorem add the fact that the loss function is **convex in its first argument**:

$$R(\frac{1}{T}\sum_{i=1}^{T} h_i) \leq \frac{1}{T}\sum_{i=1}^{T} L(h_i(\boldsymbol{x}_i), y_i) + M\sqrt{\frac{2\log(1/\delta)}{T}}$$

$$R(\frac{1}{T}\sum_{i=1}^{T} h_i) \leq \inf_{h\in\mathcal{H}} R(h) + \frac{R_T}{T} + 2M\sqrt{\frac{2\log(2/\delta)}{T}}$$

where $R_T = \sum_{i=1}^{T} L(h_i(\boldsymbol{x}_i), y_i) - \min_{h\in\mathcal{H}} \sum_{i=1}^{T} L(h(x_i), y_i)$

Conclusion

Conclusion

- Another Online algorithm for linear classifiers: Winnow with a multiplicative update $w_{t+1}, i \leftarrow w_{t,i} e^{\eta y_t x_{t,i}} / Z_t$
- Very elegant solutions to deal with large scale issues (Big Data)
- Strong guarantees, no assumption on the distribution
- SGD can be seen as a particular online learning method
- Other results can be proposed, in particular by taking into account the distribution of the data and/or particular regularizations
- Many algorithms can be transformed in an online procedure (SVM, metric learning, ...)
- There are links with Bandits and Reinforcement Learning
- Other Related area
  - Game theory
  - Online (convex) optimization